

From Black Boxes to Open Formats

The Audio Engine Behind Open Generative Studios

A technical report on the audio-rendering architecture of Open Generative Studios

Engineering Report Prepared by Mohamed Yassine Khlif — April 2026

Abstract

Generative music systems increasingly rely on deep autoregressive or diffusion models that synthesize audio directly from raw waveforms or spectrograms, an approach that is expressive but GPU-bound, non-reproducible, and largely opaque to inspection or musical editing. This report documents an alternative architecture built for Open Generative Studios: a symbolic pipeline that generates MIDI using lightweight, CPU-only statistical models (random forests for expressive humanization, Markov chains for melodic progression) and renders that MIDI to audio through an open, text-based sample format (SFZ) rather than a licensed plugin standard. We describe the evaluation that led from raw-audio generation to a MIDI-based pipeline, compare three candidate rendering backends (VST, VST3, and SFZ) and three candidate sample libraries (Spitfire Labs, Kontakt, and the Virtual Playing Orchestra) against the constraints of a stateless, containerized, horizontally scaled microservice, and present the resulting stack: a JUCE-based rendering workstation driving the open-source sfizz engine. We show that this architecture satisfies five explicit design objectives (inspectability, CPU-only operation, modularity, reproducibility, and license-free deployability) at a measurable and deliberate cost in sonic richness relative to commercial, DAW-oriented alternatives. We close by extracting design principles that we believe generalize to other teams building headless, licensing-constrained generative-audio infrastructure, and by outlining two directions for recovering fidelity without abandoning the architecture's operational guarantees.

Keywords: symbolic music generation, MIDI, sample-based synthesis, SFZ, sfizz, JUCE, headless audio rendering, reproducibility, CPU-only inference, open-source sample libraries

1. Introduction

1.1 Motivation

Turning a generative model's output into something a listener can actually hear is often treated as an implementation detail, secondary to the modeling work that produces the underlying musical content. In practice, for any system that must run continuously, deterministically, and cheaply on commodity infrastructure, the rendering stage is not secondary at all: it determines whether the system can be deployed as a stateless microservice, whether its output can be reproduced identically on request, and whether the operator depends on infrastructure they cannot fully control, a GPU fleet, a licensed digital audio workstation, or a fleet of dongled plugin activations. Open Generative Studios was built under exactly these constraints: a CPU-only server environment, a containerized deployment model, and no dependency on a licensed DAW or a GPU. This report documents the engineering path taken to satisfy them, from the earliest exploration of raw-audio generation through to the sample-library decisions that ultimately shaped the sound of the finished pipeline.

1.2 Design Objectives

The modeling and rendering decisions described in this report were not made independently. each was evaluated against the same five objectives, stated explicitly here because they recur as evaluation criteria throughout the report:

1. **Inspectability.** Every stage of the pipeline should produce artifacts a human, not just a machine, can read, audit, and edit: a MIDI file, a plain-text instrument definition, a transition matrix.
2. **CPU-only operation.** No stage may require a GPU, either for training or for inference, since the target deployment environment provides neither.
3. **Modularity.** Rendering must be cleanly separable from generation, so either stage can evolve, scale, or be replaced independently within the surrounding service pipeline.
4. **Reproducibility.** Given the same seed and inputs, the system should regenerate byte-identical or near-identical output, indefinitely, without depending on the mutable state of an interactive host application.

5. **License-free deployability.** No stage may depend on per-machine or per-seat activation schemes that are structurally incompatible with ephemeral, horizontally scaled containers.

1.3 Structure of This Report

Section 2 describes why raw-audio generation was evaluated and rejected in favor of a symbolic (MIDI) representation.

Section 3 covers the two lightweight statistical models used to add expressive detail to that MIDI.

Section 4 evaluates three candidate rendering backends against the deployment constraints above.

Section 5 describes the two open-source components that implement the chosen backend.

Section 6 evaluates three candidate sample libraries.

Section 7 synthesizes the resulting stack against the five design objectives.

Sections 8 and 9 discuss limitations and future work.

Section 10 extracts lessons we believe generalize beyond this particular system.

2. Background: Raw-Audio Versus Symbolic Music Generation

2.1 The Raw-Audio Generation Paradigm

The most direct way to generate music with a neural model is to generate the audio itself, a waveform or spectrogram that can be played back with no further processing. This is the approach taken by commercial and research systems such as Suno, Udio, and Meta AI's MusicGen, a single-stage transformer language model that operates over compressed discrete audio tokens and can be conditioned on text description or melody [Copet et al., 2023]. Raw-signal generation has two structural advantages over any pipeline that first commits to a symbolic representation. First, it has no dependency on an instrument, a sample library, or a synthesis engine: the model produces finished audio directly, with nothing left to render. Second, it can capture timbral and performative detail (breath noise, bow attack, room tone, the micro-variation of a real performance) that a symbolic pipeline can only approximate afterward, through modeling or careful sample selection.

2.2 Why Raw-Audio Generation Was Rejected

Against these advantages, three drawbacks proved decisive for this project's deployment context. First, and most fundamentally, competitive raw-audio generation requires deep autoregressive or diffusion architectures, and both classes of model are GPU-intensive to train and, more importantly for a production system, GPU-intensive to run at inference time, directly conflicting with the CPU-only server constraint. Second, raw audio is an opaque artifact: there is no note-level view into what the model produced, no mechanism for a human to correct a single wrong pitch without regenerating the whole passage, and no guarantee that the same prompt and seed will reproduce the same output across model versions or hardware. Third, the system had a hard requirement to produce an exportable score, something a musician could read, print, or import into notation software, which raw audio cannot provide by construction, whatever its fidelity.

2.3 Adopting MIDI as an Intermediate Representation

Choosing MIDI as the intermediate representation resolves all three problems at once: it is CPU-cheap to generate, fully inspectable and editable at the note level, deterministic to reproduce, and trivially exportable as a score. The cost is architectural rather than computational: MIDI is silent. Something must still turn a MIDI file into audio a listener can hear, and that rendering stage becomes a full engineering problem in its own right, addressed in Sections 4 and 5. This trade-off, controllability and transparency in exchange for an additional rendering stage, is the single structural decision that shapes the rest of the pipeline described in this report, and, we suspect, any comparable system that shares the same reproducibility and inspectability requirements.

3. Statistical Modeling for Expressive Performance

3.1 Random Forests for Velocity and Micro-Timing

With MIDI fixed as the representation layer, the next question was which model family could add convincing expressive detail, the velocity and micro-timing variation that separates a "humanized" performance from a mechanically quantized one, without reintroducing a GPU dependency. Random forests [Breiman, 2001], ensembles of decision trees trained on engineered musical features, met this requirement well.

They train quickly on CPU, produce near-instant inference (important for a pipeline meant to respond to a user prompt in close to real time), and are interpretable: feature importances show directly which musical descriptors (metric position, preceding velocity, phrase boundary, and similar engineered features) drive a given prediction. Their limitation is equally direct: a random forest models local, feature-based relationships and cannot capture long-range musical structure or learn representations end-to-end from raw data the way a neural sequence model would. Every feature it uses has to be hand-engineered rather than learned.

3.2 Markov Chains for Melodic Progression

Melodic progression is handled by a separate, simpler model family: Markov chains, specifically third-order transition matrices trained per musical style. Markov chains are computationally trivial to train and sample from, and, critically for the reproducibility objective stated in Section 1.2, fully deterministic given a fixed seed. Conditioning on genre is a matter of training a separate transition matrix per style rather than a more complex conditioning mechanism. The cost of this simplicity is memory: a third-order chain has a short window, so only the preceding three notes influence the next, which means generated passages can drift or lose thematic coherence over longer spans, and the model has no mechanism for understanding harmonic context the way an attention-based architecture would.

3.3 Summary of the Modeling Trade-off

Both model choices make the same trade: generative sophistication for speed, interpretability, and CPU-only operation. Neither is competitive with a modern neural sequence model on raw output quality in isolation, but both satisfy the inspectability and reproducibility objectives that a deep model would put at risk, and both run comfortably within the compute budget of a CPU-only microservice, precisely the trade the project's five design objectives call for.

4. Evaluating Rendering Backends: VST, VST3, and SFZ

Once MIDI generation was in place, the pipeline needed a way to turn that MIDI into audio without a GPU, without an interactive DAW, and without per-machine licensing that a

container orchestrator would routinely violate by destroying and redeploying instances on demand. We evaluated three plugin and instrument formats against these constraints.

4.1 VST

Steinberg's Virtual Studio Technology (VST) is the original and still most widely supported plugin format, with a large, mature ecosystem of instruments and near-universal compatibility with commercial DAWs. That same maturity is also the source of its unsuitability for headless server deployment: VST is a closed binary plugin format designed around interactive use inside a DAW, many instruments assume a GUI thread is available even when nothing is displayed, and the licensing and authorization schemes built around commercial VST instruments, hardware dongles, machine-locked activation, actively resist the stateless, horizontally scaled deployment a microservice pipeline requires.

4.2 VST3

VST3 is Steinberg's redesigned, more modular successor, with real technical advantages: better CPU efficiency through per-bus audio processing, cleaner parameter automation, improved sandboxing, and an SDK that is at least partially open, dual-licensed under GPL and a commercial license. It remains, however, fundamentally built for host and DAW integration. Hosting VST3 headlessly requires building or maintaining a custom plugin host, and most commercial orchestral instruments built for VST3 still carry the same per-machine licensing as their VST predecessors, a poor fit for containers that may be destroyed and redeployed on demand.

4.3 SFZ

SFZ is a different kind of artifact altogether: an open, plain-text format for describing how a set of audio samples maps to keys, velocities, and articulations, created originally for Cakewalk and now a de facto open standard maintained by the community around it. Because an SFZ definition is plain text, it is directly inspectable, satisfying the project's transparency objective in a way no binary plugin format can, and it carries no licensing lock-in and no GUI requirement. An SFZ file and its associated samples can be dropped into a Docker image and rendered by a compatible engine with no interactive host at all. The trade-off is sonic: the format itself carries none of the rich per-instrument DSP (built-in convolution reverb, advanced legato engines, deep articulation switching) that flagship commercial VST instruments bundle. An SFZ-based engine has to be paired with good samples and, where needed, external post-processing to recover some of that polish.

4.4 Comparative Summary

The migration path we followed, from VST3 to SFZ, is really a migration from "the richest possible sound, at the cost of host complexity and licensing friction" to "good enough, fully automatable sound, at the cost of some sonic sophistication." Table 1 summarizes the comparison, for a headless, GPU-free, horizontally scalable pipeline, we judge the second trade-off to be the correct one.

Table 1. Comparison of candidate rendering backends.

Dimension	VST	VST3	SFZ
Headless / server friendly	Poor	Moderate (needs custom host)	Native fit
Licensing friction	High	Moderate to high	None (open format)
Sound engine richness	High	High	Moderate
Reproducibility	Depends on host state	Depends on host state	High (plain text, deterministic)
Ecosystem size	Very large	Large	Smaller, growing

5. System Implementation

5.1 JUCE: The Internal Music Workstation

JUCE is a C++ cross-platform application framework commonly used to build both audio plugins and standalone audio applications. It underpins the internal "music workstation" microservice responsible for audio buffering, MIDI input and output, and real-time signal flow ahead of final rendering. JUCE brings mature real-time audio buffer management, genuinely cross-platform builds (useful should the project's Droplet and App Service deployment environments ever diverge), and the native ability to host plugin formats directly, which keeps a hybrid VST3/SFZ rendering path (Section 9.1) available without a rewrite. The cost is a substantial C++ dependency with a real learning curve, and licensing that is not simply "open source": the framework is dual-licensed, available under the

AGPLv3 or under a tiered commercial license (Personal, Indie, Pro, and Educational, under the JUCE 9 agreement introduced in June 2026) once revenue or distribution thresholds are crossed. This is not a static fact to take on faith: the open-source terms have themselves shifted across major versions, from GPLv3 in earlier releases to the stricter AGPLv3 today, so it is worth re-verifying against current terms before the service scales commercially rather than assuming today's terms will hold indefinitely.

5.2 sfizz: The SFZ Rendering Engine

sfizz is the open-source engine that actually turns an SFZ instrument definition and a stream of MIDI into audio. Developed by the SFZ Tools community under a permissive BSD-2-Clause license (permissive enough to be embedded in proprietary software without restriction), sfizz is lightweight, embeddable, has no GUI dependency, and can be driven entirely programmatically, which matters when the pipeline needs to trigger rendering automatically once MIDI post-processing completes. This is precisely the profile a stateless, Docker-based microservice needs. Its limitation is maturity: as a smaller, community- and donation-funded project relative to a decades-old commercial sampler, its built-in feature set (effects processing, articulation-switching logic) is less extensive than something like Kontakt, so some of that polish has to be recovered either in the SFZ instrument definitions themselves or in a post-processing step downstream of rendering.

6. Sample Library Selection

The rendering backend determines how audio is produced; the sample library determines what it sounds like.

We evaluated three libraries as the actual sound source for the pipeline:

6.1 Spitfire Labs

Spitfire Labs offers genuinely high production quality for a free product and is well suited to quick prototyping. It was disqualified for production use for reasons that have nothing to do with sound quality: it ships as its own proprietary plugin rather than as an SFZ file, its free catalog rotates over time in a way that breaks reproducibility for anything rendered months or years apart, and it is simply not designed for headless automation, a hard requirement for a server pipeline that must render deterministically on demand.

6.2 Kontakt

Kontakt, from Native Instruments, is the industry-standard orchestral sampler and sits at the center of by far the deepest and highest-fidelity third-party library ecosystem of the three options considered. Its proprietary NKI format, wrapped inside the Kontakt engine, requires per-machine license activation and a plugin architecture built for interactive DAW use, all of which work directly against a containerized, GPU-free, freely redistributable deployment model, for the same structural reasons VST and VST3 instruments were found unsuitable in Section 4.

6.3 Virtual Playing Orchestra

The Virtual Playing Orchestra (VPO), an orchestral sample library assembled by Paul Battersby from Sonatina Symphonic Orchestra and other freely available sources, is distributed at no cost, including for commercial use, directly in SFZ-compatible form. It drops straight into sfizz with no licensing negotiation, no activation server, and full automation, which made it the natural fit for the final pipeline. The cost is audible: VPO is noticeably lower fidelity than Kontakt-based commercial libraries, with fewer round-robins, fewer velocity layers, and generally a single microphone position, a direct sonic cost of choosing openness and automatability over polish.

6.4 Comparative Summary

Table 2. Comparison of candidate sample libraries.

Dimension	Spitfire Labs	Kontakt	Virtual Playing Orchestra
Format	Proprietary plugin	Proprietary (NKI, via Kontakt engine)	Open (SFZ)
Licensing / activation	Free, but rotating catalog	Per-machine activation	Free, incl. commercial use
Headless automation	Not supported	Not supported	Native fit (via sfizz)
Long-term reproducibility	Low (catalog rotates)	Constrained by per-machine activation	High (static files)

Sonic fidelity	High for a free product	Highest of the three	Moderate
-----------------------	-------------------------	----------------------	----------

7. Architectural Synthesis: Mapping the Stack to Design Objectives

The final rendering stack, a JUCE-based workstation microservice feeding a sfizz engine loaded with SFZ instrument definitions drawn from the Virtual Playing Orchestra and compatible free libraries, is best evaluated not in isolation but against the five objectives stated in Section 1.2. Table 3 makes that mapping explicit.

Table 3. Design objectives and how the rendering stack satisfies them.

Objective	How the stack satisfies it	Trade-off accepted
Inspectability	SFZ instrument definitions and MIDI files are plain text / structured data, not opaque binary plugins	None significant
CPU-only operation	sfizz and the surrounding JUCE workstation run entirely on CPU; no stage requires a GPU	Some latency versus GPU-accelerated synthesis
Modularity	Rendering is a distinct microservice boundary within the surrounding Staged Event-Driven Architecture (SEDA) pipeline [Welsh et al., 2001], separable from the generative stages	Added inter-service communication overhead
Reproducibility	Deterministic rendering with no dependency on interactive host state	None significant
License-free deployability	SFZ, sfizz (BSD-2-Clause), and VPO carry no per-seat or per-machine licensing	JUCE itself still requires a commercial license past certain revenue/distribution thresholds

The one objective this stack does not satisfy unconditionally is the last: JUCE's own licensing (Section 5.1) means "license-free" applies cleanly to the rendering path's audio content (SFZ, sfizz, VPO) but not to the framework hosting it, a distinction worth keeping

explicit rather than glossing over. Beyond that caveat, the stack is coherent: it is inspectable, CPU-only, modular, reproducible, and deployable without the per-seat licensing overhead that disqualified VST-, VST3-, and Kontakt-based alternatives. What it deliberately sacrifices, consistently across every decision described in Sections 4 through 6, is the top-tier sonic richness that Kontakt-grade libraries and full VST3 DSP chains provide.

8. Limitations

The limitation running through this report is singular and consistent: sound quality is the direct, quantifiable cost of the architectural choices made in service of automatability. A third-order Markov chain is less coherent over long passages than an attention-based model. A random forest cannot learn representations a neural sequence model would discover on its own. sfizz's built-in effects are thinner than Kontakt's. VPO's round-robins and mic positions are fewer than a flagship commercial library's. None of these are incidental engineering shortfalls, each is the predictable consequence of optimizing for CPU-only operation, license-free deployment, and reproducibility over peak fidelity. We think it is more useful to state that trade-off plainly than to treat it as a defect to be quietly fixed later.

9. Future Directions

9.1 A Hybrid Rendering Path

SFZ and sfizz can remain the default headless renderer while an optional VST3 rendering path (via a custom JUCE-based headless host) is offered to users willing to trade deployment simplicity for higher fidelity. Because JUCE already has native plugin-hosting capability (Section 5.1), this extension does not require replacing the workstation microservice, only extending it.

9.2 Expanding Sample-Library Coverage

The SFZ-compatible sample library set can be expanded beyond VPO to cover non-Western instrumentation (Middle Eastern, Arabic, and Andalusian instruments among them), which would also address a cultural-coverage gap already identified in the project's training

corpora. Because SFZ is an open, well-documented format, this is primarily a library-sourcing and curation problem rather than an engineering one, and does not require any change to the rendering architecture described in this report.

10. Generalizable Lessons for Similar Systems

While the decisions above were made for one specific project, we believe several of the underlying principles generalize to other teams building headless, license-constrained generative-audio infrastructure:

1. Prefer an intermediate symbolic representation whenever reproducibility and editability matter more than raw timbral nuance. Any system that needs to guarantee identical output on demand, or hand a human an editable artifact, pays for that guarantee with an extra rendering stage. An opaque raw-audio model cannot offer the guarantee at all, at any cost.
2. In constrained-compute, production settings, classical statistical models often win on the axes that matter operationally, even when they concede peak generative quality. Interpretability, near-instant CPU inference, and deterministic output are frequently more valuable in a production pipeline than the marginal quality gain of a deep sequence model, particularly when that gain comes with a GPU dependency the deployment target cannot satisfy.
3. Plugin ecosystems built for interactive, single-machine use resist headless containerization by design, not by accident. GUI-thread assumptions and per-machine or per-seat licensing are structural properties of formats like VST and VST3, not incidental limitations. Teams should expect to either build a custom headless host or move to a format designed for automation from the outset, such as SFZ.
4. A dedicated rendering microservice, decoupled from the generative modeling layer, lets the rendering backend evolve independently. The hybrid VST3 path proposed in Section 9.1 is only possible because rendering was never entangled with generation in the first place. The same modularity that satisfies this project's objectives also keeps future migration cheap.
5. Output quality is frequently the direct, quantifiable cost of openness and automatability, and is worth stating explicitly rather than treating as an incidental shortfall. Every open, license-free component evaluated in this report (SFZ relative to VST3, VPO relative to Kontakt) traded fidelity for deployability in a way that was measurable and predictable in advance, not a surprise discovered after deployment.

11. Conclusion

Open Generative Studios' audio engine is the product of five explicit constraints (inspectability, CPU-only operation, modularity, reproducibility, and license-free deployability) applied consistently across two separate engineering decisions: which models generate musical content, and which format renders it to sound. In both cases, the more sophisticated, higher-fidelity option (deep raw-audio generation, VST3 or Kontakt-based rendering) was evaluated and set aside in favor of a lighter, more constrained, more transparent alternative: MIDI generated by random forests and Markov chains, rendered through SFZ via sfizz and the Virtual Playing Orchestra, that could actually satisfy the deployment target. The resulting system is not the best-sounding generative music pipeline that could be built. It is, we argue, the best-sounding pipeline that can be built within the stated constraints, and making that trade-off explicit, rather than incidental, is the central contribution of this report.

References

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., & Défossez, A. (2023). Simple and Controllable Music Generation. In *Advances in Neural Information Processing Systems* (NeurIPS 2023). arXiv:2306.05284.

JUCE (Raw Material Software Limited). JUCE 9 End User Licence Agreement. juce.com/legal.

sfizz / SFZ Tools. sfizz: an open-source SFZ parser and synthesis library, licensed under BSD-2-Clause. github.com/sfztools/sfizz.

Virtual Playing Orchestra, created by Paul Battersby. virtualplaying.com.

Welsh, M., Culler, D., & Brewer, E. (2001). SEDA: An Architecture for Well-Conditioned, Scalable Internet Services. *Proceedings of the 18th ACM Symposium on Operating Systems Principles* (SOSP '01), 230–243.